Module - 2

1. Explain the C data types OR AVR C with examples?

Ans: AVR C uses standard C data types

Data Type	Size	Example
char	1 byte	char ch = 'A';
int	2 byte	int a = 1000;
unsigned int	2 byte	unsigned int x = 50000;
long	4 byte	long num = 123456L;
Float	4 byte	Float Pi = 3.14;
double	4 bytes	double d = 10.25;
void	no storage	void main(void)

2. List the three ways to create a time delay in AVR C. Explain with examples

A. 1. Software Delay (For loop)

```
For (i=0; i<5000; i++);
```

- simple method
- Less accurate

2. Predefined Delay function

```
#include <util/delay.h>
-delay_ms (100);
-delay_us (50);
```

- Accurate for short delays
- Requires F_CPU definition

3. Timer Delay

```
TCNT0 = 0;
TCCR0 = (1<<CS02);
while ((TIFR & (1<<TOV0)) == 0);
TIFR = (1<<TOV0);
```

- most accurate
- used for long delays.

3.

Ac

3. AVR C Program to toggle all bits of PORTB continuously with 100ms delay (XTAL = 8 MHz).

Ans

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

int main (void)
{
    DDRB = 0xFF;
    while (1)
    {
     PORTB = ~PORTB;
     _delay_ms (100);
    }
}
```

Ans.

```
#include <avr/io.h>
int main (void)
{
  DDRB = 0x00;
  DDRC = 0xFF;
  while (1)
  {
    PORTC = PINB;
  }
}
```

6. Explain logical operators or bitwise operators in AVR C?

Ans.

Logical operators

operator	meaning	Example
&&	Logical AND	IF (a && b)
!	Logical NOT	IF (!a)

4. AVR C Program to toggle all pins of PORTC continuously with 10ms delay using Predefined delay functions.

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
  DDRC = 0xFF;
  while(1)
  {
    PORTC = ~PORTC;
    _delay_ms(10);
  }
}
```

5 AVR C Program to get one byte of data from PORTB and send it to PORTC.

Bitwise operators

operator	meaning	example
&	Bitwise AND	A&B
^	Bitwise XOR	A^B
~	Bitwise NOT	~A
<<	Left Shift	A<<1
>>	Right Shift	A>>1

7 Explain the data types in AVR C with example

Ans unsigned char

→ size 8 bits range

→ 0 to 255 range also 00 – FFH

→ most natural choice for AVR as it's an 8-bit microcontroller

unsigned char z = 200;

signed char

→ 8-bits in size

→ -128 to +127 in range

→ uses most significant bit as sign bit

char temperature = -10;

unsigned int

→ 16-bit in range

→ 0 to 65,535 in ranges

→ used for 16-bit variables and counter values > 256

unsigned int counter = 50000;

Signed int

→ 16 bit in size
→ -32,768 to +32,767 as in range

int value = -1000;

unsigned long

→ 32 bit in size
→ 0 to 4,294,967,295 as in range.

unsigned long count = 100000;

Float and double

→ 32 bit as in size
→ used for fractional numbers

Float temp = 25.5;

9. Write an AVR C Programs to toggle all bits of PORTB continuously with a 100ms delay. Assume ATmega32 with XTAL = 8 MHz.

Ans.

```
#include <avr/io.h>  // Standard header file (AVR).
void delay100ms(void){
    unsigned int i;
    for(i=0; i<42150; i++){}
    }
int main(void){
    DDRB = 0xFF;  // PORTB as output
    while (1){
        PORTB = 0xAA;
        delay100ms();
        PORTB = 0x55;
        delay100ms();
    }
return 0;
}
```

10.

8. List and explain the three ways to create time delays in AVR C with examples?

Ans. 1. Software Delay (For loop)

For (i=0; i<5000; i++);

→ Simple method

→ Less accurate

2) Using a simple For loop

```
void delay_1000ms (void){
    unsigned int i;
    For (i=0; i<42150; i++){} // different numbers.
}
```

3) Using Predefined C Functions

```
#include <util/delay.h>
void delay_ms(int d){
    _delay_ms(d); // in AVR Function
```

10. write an AVR C program to get a byte of data from PORTB and send it continuously to PORTC

Ans.

```
#include <avr/io.h> //standard AVR header

int main (void){

    unsigned char temp;

    DDRB = 0x00;        // Port B as input

    DDRC = 0xFF;        // Port C as output

    while (1){

        temp = PINB;    // Read from PORTB

        PORTC = temp;   // send to PORTC

    }

return 0;

}
```

11. Explain logical/ bitwise operations in AVR C with examples?

Ans. AND operation (&)

used to clear specific bits masking

PORTB = PORTB & 0b11101111; // clear bit 4

OR operation (|)

used to sets specific bits

PORTB = PORTB | 0b00010000; // set bit 4

XOR operation (^)

used to toggle bits

PORTB = PORTB ^ 0b00010000; // Toggle bit 4

NOT operation (~)

used to invert all bits

PORTB = ~PORTB; // Toggle all bits

Shift operations (<<)

(1<<7) // number with one in D7

~(1<<5) // number with zero in D5

15. Write an AVR C program to monitor bit 5 of PORTC. If it is HIGH, send 55H to PORT B; otherwise, send AAH to PORT B.

Ans.

```
#include <avr/io.h>                    //Standard AVR header
int main (void)
{
   DDRB = 0xFF;                        // PORTB is output
   DDRC = 0x00;                        // PORTC is input
   DDRD = 0xFF;                        // PORTB is output
   while (1)
   {
      IF (PINC & 0b00100000)           // check bit5 of PINC
         PORTB = 0x55;
      else
         PORTB = 0xAA;
   }
   return 0;
}
```

12 write an AVR C program to toggle only 4th bit of PORTB continuously without disturbing the other bits.

Ans

```
#include <avr/io.h>          // standard AVR header

int main (void)
{
  DDRB = 0xFF;               // PORTB is output
  while (1)
  {
    PORTB = PORTB | 0b00010000; // set bit 4 of PORTB
    PORTB = PORTB & 0b11101111; // clear bit 4 of PORTB
  }
return 0;
}
```

14 Write AVR C program to toggle all pins of PORTB using the NOT operator and XOR operator.

Ans.

```
#include<avr/io.h>
#include<util/delay.h>

int main(void)
{
    DDRB = 0xFF;        // Configure PORTB as output
    PORTB = 0x00;       // Initial value
    while(1)
    {
        PORTB = ~PORTB;  // Toggle using NOT operator
        _delay_ms(500);
        PORTB ^= 0xFF;   // Toggle using XOR operator
        _delay_ms(500);
    }
}
```

15. write an ARM C program to convert packed BCD 0x29 to ASCII and display the ASCII bytes on PORTB and PORTC

Ans

```
#include <avrlibio.h>          //standard AVR header.
int main (void)
{
  unsigned char x, y;
  unsigned char mybyte = 0x29;

  DDRB = DDRC = 0xFF;          //make ports B and C output
  x = mybyte & 0xF0;           //make upper 4 bits
  PORTB = x | 0x30;            //make it ASCII
  y = mybyte & 0x0F;           //mask lower 4 bits
  y = y >> 4;                  //shift it to lower 4 bits

  PORTC = y | 0x30             //make it ASCII

  return 0;
}
```

16 Write an AVR C program to convert ~~Packed~~ ASCII digits '4' and 'C' to packed BCD and display the result on PORTB.

By.

```
#include<avr/io.h>          //standard AVR header

int main(void)

{
  unsigned char bcdbyte;
  unsigned char w = '4';
  unsigned char z = 'C';
  DDRB = 0xFF;              //make PORT B an output
  w = w & 0x0F;             // mask 3
  w = w << 4;               //shift left to make upper BCD digit
  z = z & 0x0F              //mask 3
  bcdbyte = w | z;          //combine to make Packed BCD
  PORTB = bcdbyte;
  return 0;
}
```

Q7 Write an AVR C program to read a byte from PORTB. If it is less than 100, send it to PORTC; otherwise send it to PORTD.

Ans.

```
#include<avr/io.h>          //standard AVR header

int main(void){

    DDRB = 0x00;            // PORTB as input
    DDRC = 0xFF;            // PORTC as output
    DDRD = 0xFF;            // PORTD as output

    unsigned char temp;
    while (1){
        temp = PINB;        // Read from PORTB
        if (temp <100){
            PORTC = temp;
        }else{
            PORTD = temp;
        }
    }
    return 0;
```

Q8: predict the value of PORTB, PORTC, and PORTD after executing; PORTB = 0x35 & 0x0F; PORTC = 0x04 | 0x60;

PORTD = 0x54 ^ 0x78;

Ans. -> PORTB = 0x35 & 0x0F

0x35 = 0011 0101

0x0F = 0000 1111

&

0000 0101 = 0x05 ∴ PORTB = 0x05

-> PORTC = 0x04 | 0x60

0x04 = 0000 0100

0x60 = 0110 0000

|

0110 0100 = 0x64 ∴ PORTC = 0x64

-> PORTD = 0x54 ^ 0x78

0x54 = 0101 0100

0x78 = 0111 1000

^

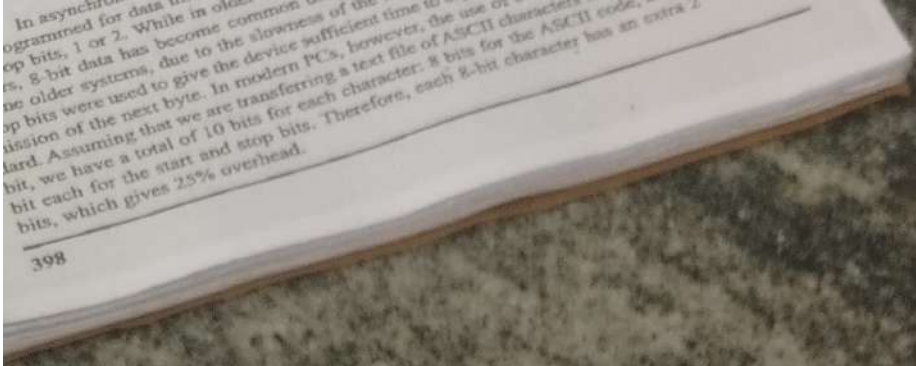
0010 1100 = 0x2C ∴ PORTD = 0x2C

19. write an AVR C programs to send the hexadecimal ASCII codes OR characters 0, 1, 2, 3, 4, 5, A, and B to PORTB.

```
#include <avr/io.h>          //standard the header

int main (void)
{
    unsigned char myList[] = "012345AB";
    unsigned char z;

    DDRB = 0xff; // PORTB is output
    for (z=0; z<8; z++) {
        PORTB = myList[z]; //Add delay to observe output
    }
    while (1);
    return 0;
}
```



20. Explain the basic registers associated with AVR timers. (TCNTn, OCRNn, TCCRn, TIFRn, TIMSKn)

Ans :-

1. TCNTn (Timer/counter register).

→ TCNTn stands for Timer/counter register.

→ It stores the current value of the timer or counter

→ The value increases with every timer clock pulse.

→ When the timer reaches its maximum value, it overflows and starts counting again from zero.

eg:- TCNT0 = 100; loads the timer0 counter with the value 100.

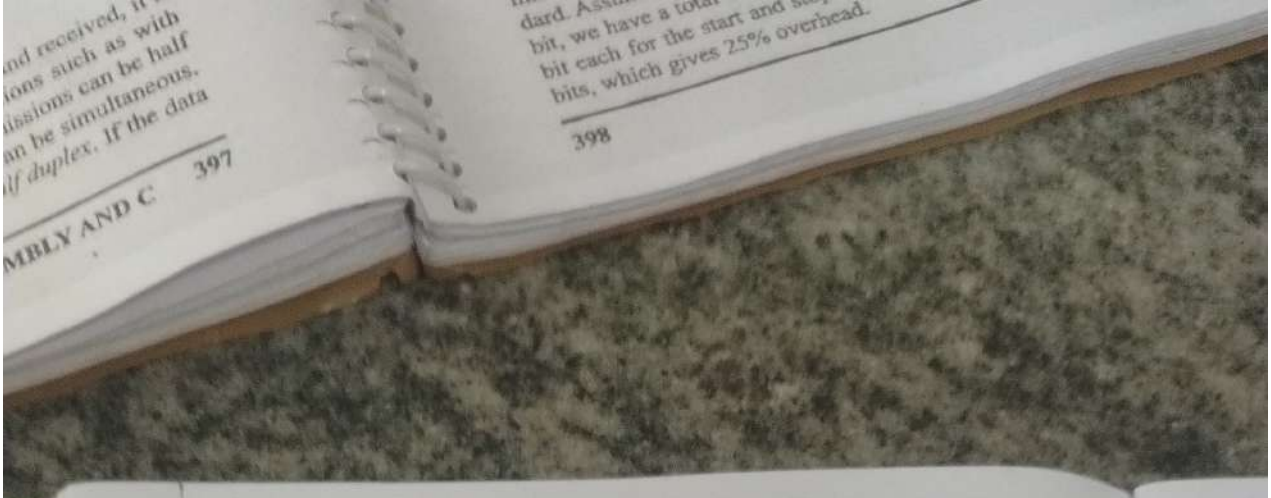
2. OCRn (Output compare Register)

→ OCRn stands for output compare register

→ It stores the value that is compared with the current value of TCNTn

→ When TCNTn = OCRn a compare match occurs.

→ This compare match can generate an interrupt toggle an output pin, or reset the timer depending on the selected mode



→ It is mainly used in CTC mode, PWM generation, and waveform control.

3. TCCRn (Timer/Counter control register).

→ TCCRn stands for Timer/Counter control register

→ It controls the overall operation of the timer.

→ It is used to :-

- Select the timer clock source
- Set the prescaler value
- Choose the timer operating mode
- Configure output compare behavior.

4. TIFRn (Timer interrupts flag register).

→ TIFRn stands for Timer interrupts Flag Register.

→ It contains status flags that indicate timer events

→ The hardware automatically sets these flags when timer events occur.

→ Flags are cleared by writing a logic 1 to the corresponding bit.

Common Flags

- TOV0 - Timer overflow Flag.
- OCF0 - Output compare match Flag
- ICF0 - Input capture Flag (available in some timers

5. TIMSK0 (Timer interrupts mask register)

→ TIMSK0 stands for Timer interrupts mask Register
→ It enables or disables timer interrupts
→ setting a bit to 1 enables to the corresponding interrupts
→ Clearing a bit to 0 disables the interrupts.

Interrupts enables Bits :-

- TOIE0 - Timer Overflow interrupt Enables
- OCIE0 - Output compare match interrupt Enables
- ICIE0 - Input capture interrupts Enable

and received, it is
ssions such as with
missions can be half
can be simultaneous.
alf duplex. If the data

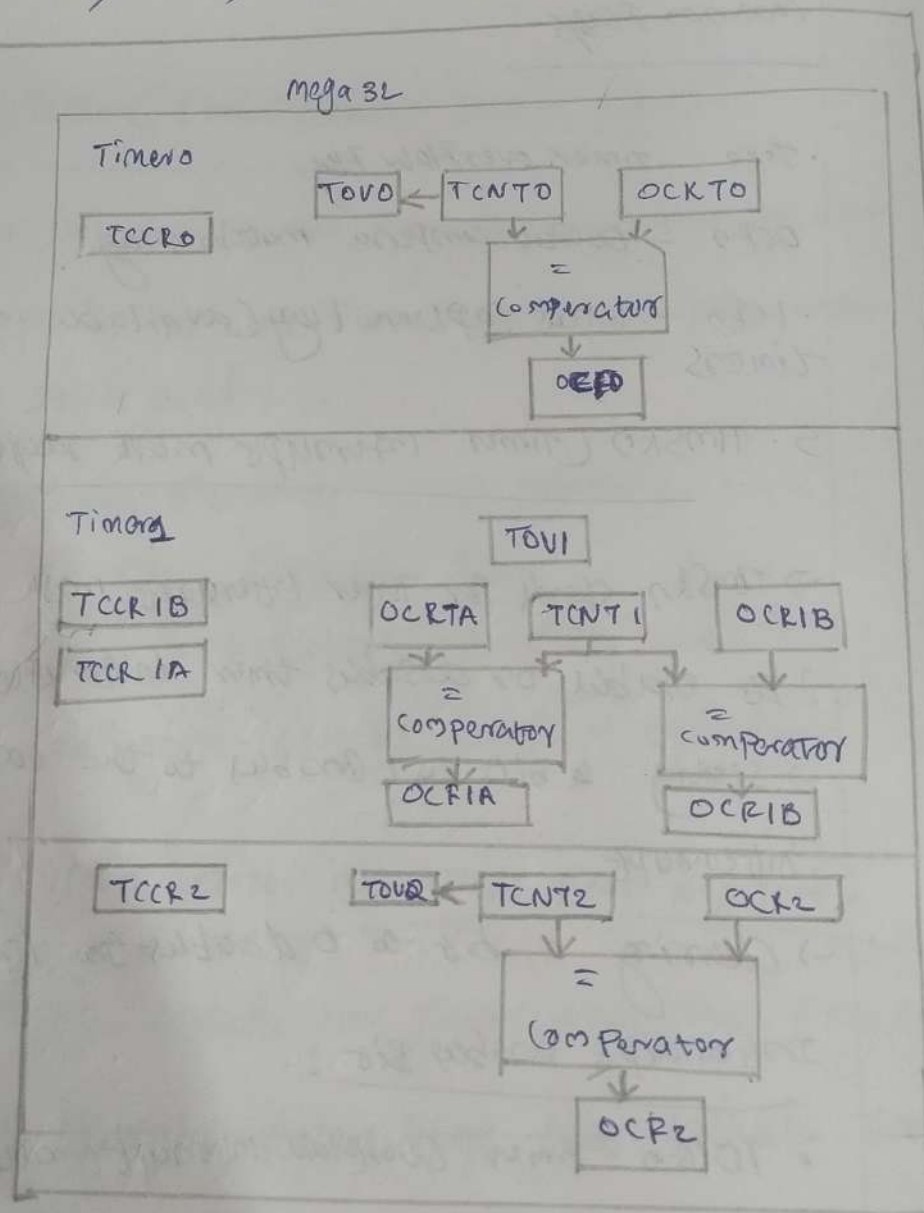
EMBLY AND C 397

mission of
dard. Assuming
bit, we have a total of 10
bit each for the start and stop bi
bits, which gives 25% overhead.

398

21 Draw and explain the block diagrams of a Timer0, Timer1, and Timer2.

Ans



Timer0

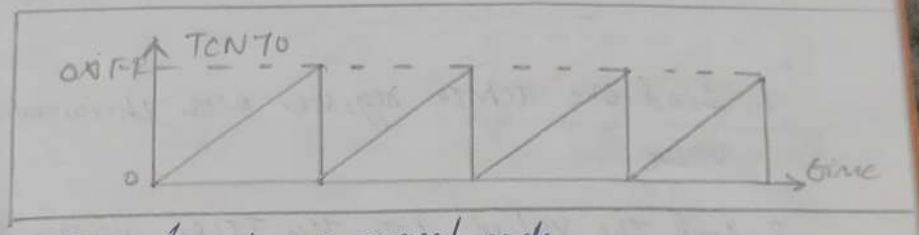
Timer0 is an 8-bit timer/counter in the Atmega32 microcontroller used for delay generation, event counting, and PWM applications. It receives the CPU clock through a clock select circuit, which allows the selection of different clock sources or prescalar values. The prescalar divides the clock frequency before it reaches the 8-bit Timer/Counter register, which counts from 0 to 225. The timer continuously compared with the OCR0 (Output Compare Register) value. When both values become equal, a compare match occurs, setting the output compare flag. When the counter overflows from 225 to 0, the Timer overflow flag (TOV0) is set which can also generate an interrupt.

Timer1

Timer1 is a 16-bit timer/counter designed for high-precision timing, PWM generation, frequency measurement, and input capture applications. It receives the CPU clock through a clock select circuit

22. Explain Timer 0 Normal mode with a neat diagram

Ans.



Timer/Counter 0 Normal mode.

In Normal mode Timer 0 operates as an 8-bit up-counter. The Timer recives the clock signal from the CPU through the clock selects bits (CS02-CS00) and an optional prescaler, which divides the clock frequency before it reaches the timer. The Timer/Counter Register (TCNT0) starts counting from the value loaded into it and increments by one on every timer clock pulse until reaches 255 (0XFF). On the next clock pulse, the counter overflows flag (TOV0) in the TIFR register is set. If the timer overflow interrupt enable (TOIE0). This mode is mainly used for time delay generation, events counting, and periodic interrupts generation.

(CS12 - CS10) and a pre scalers. The timer counts from 0 to 65,535 i.e. this is 16 bit timer/counter register (TCNT1). It has two output compare Registers, OCR1A and OCR1B, which generate compare match events when the timer count equals the stored value. Timer 1 also includes an input capture unit (ICP1). An overflow flag is set when the counter exceeds 65,535 and returns to zero

Timer 2

Timer 2 is an 8-bit timer/counter similar to Timer 0 but includes support for asynchronous operation using an external 32.768 kHz crystal oscillator. To derive the clock from either the CPU or the external crystal through the clock select circuit and Prescaler. The timer counts from 0 to 255 in the Timer/counter Register (TCNT2). The count is continuously compared with the output compare register (OCR2). Because of its asynchronous capability, Timer 2 is widely used in real-time clock (RTC).

23. write the steps required to programs Timer0 in normal mode

Ans. Steps to programs Timer0 in normal mode

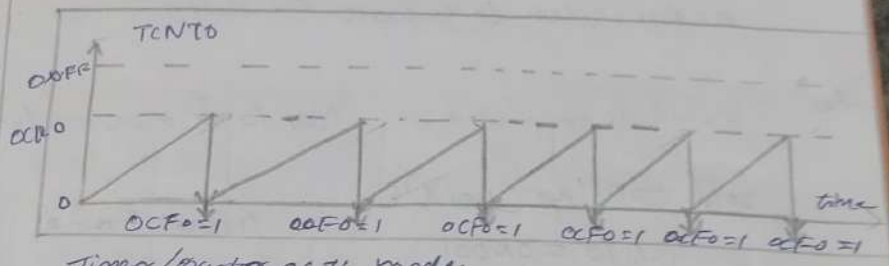
1. Load the TCNT0 register with the initial count value.
2. Load the value into the TCCR0 register, indicating which mode is to be used and the prescaler option. When you select the clock source, the timer/counter starts to count, and each tick causes the contents of the timer/counter to increment by 1.
3. Keep monitoring the timer OVERFlow Flag (TOV0) to see if it is raised. Exit out of the loop when TOV0 becomes high.
4. Stop the timer by disconnecting the clock source, using following instructions.-

```
LDI   R20, 0x00
OUT   TCCR0, R20      ; Timer stopped, Mode is NOT
```

5. clear the TOV0 Flag for the next round.
6. Go back to step 1 to load TCNT0 again.

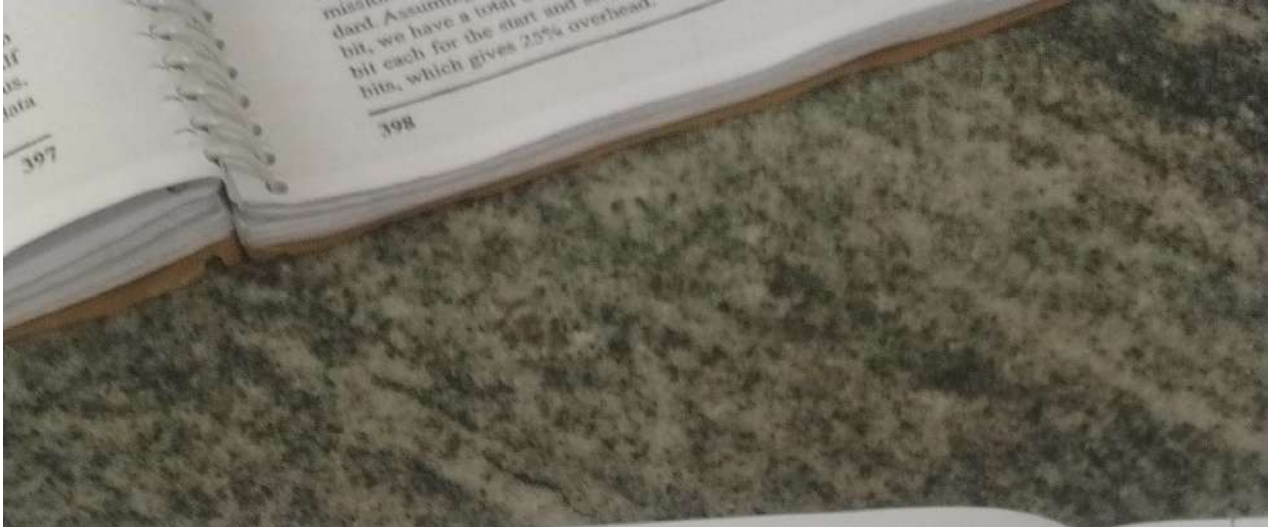
24 Explain clear Timer on compare match (CTC) mode Programming with Figure?

Ans.



Timer/Counter0 CTC mode

In clear Timer on compare match (CTC) mode, the timer counts upward from 0 and continuously compares the timer/counter register (TCNT0) with the output compare register (OCR0). When the value of TCNT0 becomes equal to OCR0, a compare flag (OCF0) is set, and the timer is automatically cleared to 0 on the next clock pulse instead of counting upto 255. If the output compare match interrupt enable (OCIE0) bit in the TIMSK register and the global interrupts is generated. The compare value is loaded into the OCR0 register. This mode is mainly used for accurate time delay generation, periodic interrupts, square wave generation etc. ---



22. write a c program to toggle all bits of PORTB continuously with Timer0 in normal mode with no prescaler to generate the delay.

Ans.

```
#include<avr/io.h>          standard AVR header

void toDelay(void){          //start from 0
    TCNT0 = 0x00;
    TCCR0 = 0x01;            // normal mode, no prescaler
    while((TIFR & (1<<TOV0)) == 0);   //wait for overflow
    TCCR0 = 0x00;            // stop timer
    TIFR = (1<<TOV0);        // clear Tov0 flag
}

int main(void){
    DDRB = 0xFF;             //PORTB as output
    PORTB = 0x55;
    while(1){
        PORTB = ~PORTB;      // Toggle
        toDelay();           //delay
    }
    return 0;
}
```

26.

Ans.

26. write a program to toggle all bits of PORTB continuously using Timer0 in CTC mode with no prescaler. Also to generate the delay.

```
#include <avr/io.h>                    // standard AVR header

void ToDelay (void){
    TCNT0 = 0x00;
    OCR0 = 249;                        // Load compare value
    TCCR0 = 0x09                       // CTC mode, no prescaler
    while ((TIFR & (1<<OCF0)) == 0);   // wait for compare match
    TCCR0 = 0x00;                      // stop timer
    TIFR = (1<<OCF0);                  // clear OCF0 flag
}

int main (void){
    DDRB = 0xFF;                       // PORTB as output
    PORTB = 0x55;
    while (1) {
        PORTB = ~PORTB;                // Toggle
        ToDelay();                     // Delay
    }
    return 0;
}
```

(ICNC) bits for input capture operation.

4. OCR1A and OCR1B (Output compare registers)

These are 16-bit Compare registers used to store compare values. When the value in TCNT1 matches OCR1A or OCR1B, a compare match occurs, setting the compare flag and optionally generating an interrupt or controlling the OC1A/OC1B output pin.

5. ICR1 (Input capture Register):

ICR1 is a 16-bit register used in input capture mode. When an external event occurs on the ICP1 pin, the current value of TCNT1 is copied into ICR1. It is mainly used for measuring pulse width, frequency, and time intervals.

6. TIFR (Timer Interrupt Flag Register)

TIFR stores the interrupt status flags for Timer1. Important flags include TOV1 (Timer overflow flag), OCF1A/OCF1B (Output compare match flags), and ICF1 (Input capture flag). These flags are set when the corresponding timer events occur and are cleared.

27. Explain the registers associated with Timer 1?

Ans. 1) TCNT1 (Timer/counter 1 register):

TCNT1 is a 16-bit timer/counter register that stores the current count value of Timer1. It consists of 8-bit registers: TCNT1H (high byte) and TCNT1L (low byte). The timer increments or decrements with each clock pulse depending on the selected mode. It is used for timing, counting events, and delay generation.

2) TCCR1A (Timer/Counter 1 Control Register A):

TCCR1A controls the waveform generation mode (WGM) and compare output mode (COM1A and COM1B). It determines how the OC1A and OC1B output pins behave during compare match and PWM operations.

3). TCCR1B (Timer/Counter 1 control Register B).

TCCR1B contains the remaining waveform generation mode (WGM) bits, the clock select (CS12-CS10) bits for selecting the timer clock source and prescaler, and the input capture edge select (ICES1) and input capture noise canceler

18. Describe the modes of operation of Timer 0 in Atmega 32.

Ans Timer 0 in Atmega32 is an 8-bit timer/counter that supports four operating modes. These modes are selected using the waveform generation mode (WGM) bits in the TCCR0 register.

1. Normal mode.

-> Timer counts from 0x00 to 0xFF.
-> After reaching 255, it overflows to 0x00.
-> Overflow sets the TOV0 flag.
-> Used for delay generation and time measurement.

2. Clear Timer on Compare match mode (CTC)

-> Timer counts from 0x00 to the value in OCR0
-> When TCNT0 equals OCR0, timer resets to 0x00.
-> Compare Match Flag (OCF0) is set
-> Used for accurate timing and periodic interrupts.

3. Fast pwm mode

→ Generated pulse width modulation (PWM) signals.

→ Timer counts from 0x00 to 0xFF continuously

→ output frequency is high.

→ used for motor speed control and LED brightness control.

4. phase correct pwm mode:

→ Timer counts up from 0x00 to 0xFF and then down to 0x00.

→ Produces symmetrical pwm waveform

→ Gives better pwm accuracy than Fast Pwm.

→ used in motor control applications requiring low harmonic distortion.

29. Differentiate between Timer0 and Timer2

Ans

Timer0	Timer2
→ Timer 0 is an 8-bit timer / counter	Timer 2 is also an 8-bit timer / counter
→ operates using the system CPU clock or an external clock source	operable using one system clock or an independent asynchronous clock crystal
→ Does not support asynchronous operation	Support asynchronous operation, allowing it to run independently of the CPU clock
→ Controlled by the TCCR0 (Timer/Counter control Register)	Controlled by the TCCR2 (Timer/Counter control Register 2).
→ Output Compare Register is OCR0	Output Compare Register is OCR2.
→ Compare match flag is OCF0	Compare match flag is OCF2
→ Simpler configuration and widely used in embedded applications.	Slightly more complex due to asynchronous clock support but more versatile for timing applications.

30 Differentiate between polling and interrupt

polling	interrupt
→ CPU continuously checks devices status	Device informs CPU when service is needed
→ wastes CPU time	Efficient CPU utilization
→ slower response	Faster response
→ simple to implement	more complex to implement
→ CPU is always busy checking	CPU performs other tasks until interrupt occurs
→ suitable for simple applications	suitable for real-time applications
→ no interrupt hardware required	Requires interrupt hardware support

22. List the source of interrupts in AVR microcontroller.

Ans. Sources of interrupts in the AVR

1. There are at least two interrupts set aside for each of the timers, one for overflow and another for compare match.

2. Three interrupts are set aside for external hardware interrupts. Pins PD2 (PORTD2), PD3 (PORTD3), and PB2 (PORTB2) are the external hardware interrupts INT0, INT1, INT2 respectively.

3. Serial communication's USART has three interrupts one for receive and two interrupts for transmit

4. The SPI interrupts.

5. The ADC (analog-to-digital converter) etc...

31. Explain the sequence/steps involved in executing an interrupts.

A. Steps in executing an interrupts

1. It finishes the instruction it is currently executing and saves the address of the next instruction on the stack.
2. It jumps to a fixed location in memory called the interrupt vector table. The interrupt vector table directs the micro controller to the address of the interrupt service routine (ISR).
3. The micro controller starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine, which is RETI (Return from interrupts).
4. Upon executing the RETI instruction, the micro controller returns to the place where it was interrupted. First, it gets the program counter (PC) address from the stack by popping the top bytes of the stack into the PC. Then it starts to execute from that address.

33 Explain hardware interrupts with its registers

Ans. Hardware interrupts are generated by external devices or internal peripherals without CPU intervention. When an interrupt occurs, the CPU automatically executes the corresponding interrupt service routine (ISR).

Register used :-

1. GICR (General interrupt Control Register)
2. GIFR (General External Flag Register
3. MCUCR (MCU control Register).
4. MCUSSR (MCU Control Status Register
5. SREG (Status Register).

Advantages

- Fast response
- Efficient CPU utilization
- Suitable for real-time systems

8A. write an AVR C program to generate a 1 KHz square wave on PB5 (toggle) using timer0 overflow interrupt while continuously sending PORTC data to PORTD.

```
#include <avr/io.h>                 // standard AVR header
#include <avr/interrupt.h>          // interrupt header

ISR (TIMER0_OVF_vect){
    TCNT0 = -32;                    // Reload timer value
    PORTB ^= (1<<5);                // Toggle PB5
}

int main (void){
    DDRB = 0x20;                    // PB5 as output
    DDRC = 0x00;                    // PORTC as input
    DDRD = 0xFF;                    // PORTD as output
    TCNT0 = -32;                    // initial value for 1kHz
    TCCR0 = 0x01;                   // Normal mode, no prescaler
    TIMSK = (1<<TOIE0);             // Enable Timer0 overflow interrupt
    sei();                          // enable global interrupts
    while (1){
        PORTD = PINC;               // Transfer PORTC to PORTD
    }
    return 0;
}
```